DAY 05 / 11 · WEB DEV IN 11 DAYS · GUIDED

Functions and decisions

Reusable code, if and else

ShowShelf after today: **each function you finish makes all four cards light up**

TODAY

The plan for the next 90 minutes

Minutes	Topic
10	Recap: five questions about Day 4
8	Define and call
12	Parameters and arguments
15	return: give the answer back
12	Decisions inside a function
13	Shorter and smarter: reuse, arrows, defaults
5	The big idea, and questions
15	This afternoon in the Lab

RECAP · DAY 4

Five quick questions

1. What do 236 + 62 and "236" + 62 print?
2. An episode counter that goes up: const or let?
3. "Watching " + showName + " tonight" as a template literal?
4. What does querySelector("#title").textContent = showName do?
5. What's wrong with if (showName = "Friends")?

FUNCTIONS

Write it once, run it whenever you like

Define: write the recipe. Nothing happens yet.

Call: cook it. The name, then round brackets.

No brackets, no call: announce; does nothing.

```
function announce() {
  console.log("Next episode in 5 seconds...");
}

announce();
announce();
```

PARAMETERS AND ARGUMENTS

Give it something to work with

show is a parameter: an empty box in the definition.

"Dark" is an argument: what goes in the box when you call.

```
function nowPlaying(show) {
  console.log(`Now playing: ${show}`);
}

nowPlaying("Dark");
nowPlaying("Fleabag");
```

```
function introduce(show, year) {
  console.log(`${show} started in ${year}`);
}

introduce("Chernobyl", 2019);
introduce(2019, "Chernobyl");   // oops
```

RETURN

Give the answer back

console.log: says it out loud, to you

```
function totalSeats(rows, seatsPerRow) {
  console.log(rows * seatsPerRow);
}

const seats = totalSeats(10, 12);
console.log(seats);   // undefined
```

return: hands it over, to the code that asked

```
function totalSeats(rows, seatsPerRow) {
  return rows * seatsPerRow;
}

const seats = totalSeats(10, 12);
console.log(seats);   // 120
```

return also ends the function: lines after it never run.

DECISIONS INSIDE A FUNCTION

Each block returns its own answer

JavaScript checks from the top and stops at the **first** true condition.

Strictest test first. And >= when the edge counts.

```
function lengthLabel(minutes) {
  if (minutes >= 60) {
    return "Long episode";
  } else if (minutes >= 30) {
    return "Normal episode";
  } else {
    return "Short episode";
  }
}

lengthLabel(60);   // "Long episode"
lengthLabel(22);   // "Short episode"
```

TWO MORE TOOLS

Compare two parameters, split a number

```
function seasonLabel(current, total) {
  if (current === total) {
    return "Final season";
  } else {
    return `Season ${current} of ${total}`;
  }
}
```

=== asks. One **=** changes.

```
const eggs = 14;
Math.floor(eggs / 6);   // 2 full boxes
eggs % 6;               // 2 left over
```

Math.floor rounds down. **%** is the remainder: what's left over.

FUNCTIONS THAT USE FUNCTIONS

Ask another function, don't repeat it

A comparison already **is** true or false. Return it directly.

Names like **isSoldOut**, **hasPoster**, **canWatch** promise a true or false.

```
function totalSeats(rows, seatsPerRow) {
  return rows * seatsPerRow;
}

function isSoldOut(rows, seatsPerRow, sold) {
  return sold >= totalSeats(rows, seatsPerRow);
}

isSoldOut(10, 12, 120);   // true
isSoldOut(10, 12, 80);    // false
```

ARROW FUNCTIONS

A shorter way to write the same function

```
function totalSeats(rows, seatsPerRow) {
  return rows * seatsPerRow;
}

// exactly the same, as an arrow function:
const totalSeats = (rows, seatsPerRow) => rows * seatsPerRow;
```

One line: the value after => is returned

Curly brackets: write return yourself

Keep it above the lines that call it

DEFAULT VALUES

A value for when the call leaves it out

show = "something good" means: if nobody gives me a show, use this.

> An argument, when there is one, always wins.

```
function nowPlaying(show = "something good") {
  console.log(`Now playing: ${show}`);
}

nowPlaying();          // something good
nowPlaying("Dark");    // Dark
```

THE BIG IDEA

console.log says the answer out loud. return hands it over.

THIS AFTERNOON IN THE LAB

Ten steps, about three hours

Steps 1–3

Functions

1 Your first function

2 Parameters and arguments

3 Two parameters

Step 4

return

4 Give the answer back with return

Steps 5–7

Decisions

5 Decisions inside a function

6 Compare two parameters

7 Hours and minutes

Steps 8–10

Shorter, smarter

8 Functions that use functions

9 Arrow functions

10 A default value

Milestone: every grey note on the four cards is filled by your functions.

STUCK?

In this order

1

Read the message

The “Not yet” box says what to fix.

2

Console

A red line names the problem and the line number.

3

Hint

Two per step. The second is nearly the answer.

4

Answer

Compare it with yours. Find the difference.

5

Ask

Your teacher, or the person next to you.

Most of the time, steps 1 to 3 are enough. That's how developers work too.

END OF THE DAY

Before you go

1. Defining a function, calling a function: what's the difference?
2. console.log instead of return: what does the caller get?
3. Why must rating >= 9 come before rating >= 8?

FINISHED EARLY?

Extra challenges

Episode codes like S01E05 · How many days to binge it?

TOMORROW

Lists of shows

Arrays, objects and loops: all twelve shows at once.